Gmail

Jeff Lian <jerry@museder.com>

[WordPress Plugin Directory] Review in Progress: Museder RestoreOne

WordPress.org Plugin Directory <plugins@wordpress.org> 2026年5月14日 凌晨2:47
收件者: Jerry Lin <jerry@museder.com>

Your review has been successfully completed. Details on how to access your SVN repository will be emailed within 24 hours to the address from which you submitted your plugin.

What to do next

You do not need to reply to *this* email, however please read the rest of it :)

From this point, we will not re-review your code unless we're alerted to a guideline or security issue. We trust you to have the necessary skills to manage and maintain your own plugin.

1. Add plugins@wordpress.org to your address book so you don't miss any emails. It is important that we can reach you by email to notify you of any issues, changes, or news related to your plugin or the directory.
2. Follow https://make.wordpress.org/plugins for important notifications and updates
3. Upload your code to the directory via SVN

Remember to review the plugin guidelines. All plugins and developers, as well as support staff, are required to follow the guidelines.

- https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/

Please keep in mind that a successful review **does not** guarantee that your plugin is free from any and all security issues or guideline violations. We, like you, are humans. Humans make mistakes.

We recommend that you continue checking your code with tools such as PHPCS+WPCS and Plugin Check. It's in everyone's best interest to ensure that your plugin stays secure, compatible, and compliant.

Should an issue arise later, your plugin will be closed and you will be notified to fix it as soon as possible. This is standard operating procedure for all plugins (even the big ones), and we appreciate your understanding of this policy.

SVN Tips and Tricks

If you're new to SVN, it can be a little weird to get used to. Unlike common uses of repositories like Git, we use SVN as a *release* system. This means that you should only be pushing ready-to-be-used versions of your plugin.

The automated approval email that follows this will contain information as to how to use SVN. We ask you please read **all** the links in that email before asking us for help. We will not be able to give you explicit usage directions, as there are a lot of SVN clients out there and we are not experts in them all. We also cannot upload your code for you, so you will be expected to master this system.

To get started, you should read this:

https://developer.wordpress.org/plugins/wordpress-org/how-to-use-subversion/

Some common points of confusion are:

- SVN user IDs are the same as your WordPress.org login ID – not your email address
- User IDs are case sensitive (if your account is JoDoe123HAY then you must use that exact ID)
- You can **set up your SVN credentials** (if you haven't already) in the “Account & Security” section of your WordPress profile – https://profiles.wordpress.org/me/profile/edit/group/3/?screen=svn-password

- Your readme content determines what information is shown on your WordPress.org public page – https://developer.wordpress.org/plugins/wordpress-org/how-your-readme-txt-works/
- Your plugin banners, screenshots, and icons are handled via the special plugin assets folder – https://developer.wordpress.org/plugins/wordpress-org/plugin-assets/

(In order to speed up the process for everyone, please do not reply to this email unless you have a question. Thank you and good luck!)

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/

On Wed, May 6, 2026 at 6:53 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

> This is an automated message to confirm that we have received your updated plugin file.
>
> File updated by artherslin, version 2.7.261.
> Comment: Hello WordPress Plugins Team,
>
> Thank you for the review. Please find Museder RestoreOne 2.7.261. The bullets below map directly to your latest review / rejection themes: readme & external behavior disclosure, nonces / capabilities & REST, path & filesystem safety, free-tier / directory-policy perception, Plugin Check, and distribution package contents (see readme.txt + Changelog for 2.7.255–2.7.261).
>
> Readme: External services (only local wp-cron.php loopback; admin assets from assets/), Privacy, uninstall behavior, and Multisite not formally supported; FAQ clarifies third-party formats are not an endorsement.
>
> Security / paths: realpath() + directory-prefix boundaries on sensitive paths; Chunk v2 upload_id restricted to UUID v4 with safe path joins; WPress aligned.
>
> REST: v2 restore / Chunk v2 / Free AI use permission_callback + two-step wp_rest nonces (consistent since 2.7.254).
>
> Free-tier UX: Neutral optional add-on copy; primary JS global MusederRestoreOneAddon (Pro is alias-only).
>
> Package: ZIP excludes dev trees (tools/, docs/, etc.).
>
> Checks: Plugin Check 1.9.0 → no errors; smoke-tested Dashboard / Backups / Restore on a clean install (HTTP 200).
>
> If you want this mapped line-by-line to a specific review email or file/line references, tell us and we will reply point-by-point. We can also provide test steps on request.
>
> Best regards,
> Jerry Lin (artherslin)
>
> https://tw.wordpress.org/plugins/files/2025/11/06_18-53-53_museder-restoreone.zip
>
> --

WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
[隱藏引用文字]

[隱藏引用文字]

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
[隱藏引用文字]

[隱藏引用文字]

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
[隱藏引用文字]

[隱藏引用文字]

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
[隱藏引用文字]

[隱藏引用文字]

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', $options);
- add_shortcode('musere_shortcode', $callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global $musere_options;
- add_action('wp_ajax_**musere**_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of __ (double underscores), wp_ , or _ (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using _n() or __() for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using if (!function_exists('NAME')) { around all your functions and classes sounds like

a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using if-exists should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

This plugin is using the prefix "museder" for 152 element(s).

# Looks like there are elements not using common prefixes.
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.
Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.

✓ **I tested my updated plugin on a clean WordPress installation** with WP_DEBUG set to true.

> ⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ **I acknowledge that this review will be rejected** if I overlook the issues or fail to test my code.
✓ **I went to "Add your plugin" and uploaded the updated version.** *I can continue updating the code there throughout the review process — the team will always check the latest version.*
✓ I replied to **this** email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

ℹ️ To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us.** *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T11 1Apr26/3.9 (P0TDX263980HGN)

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
On Tue, Mar 24, 2026 at 3:13 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

> This is an automated message to confirm that we have received your updated plugin file.
>
> File updated by artherslin, version 2.7.246.
> Comment: Dear Plugin Review Team,
>
> Thank you for your continued review of **Museder RestoreOne**. I have uploaded **version 2.7.246**.
>
> This update specifically addresses the WordPress.org review concerns:
>
> - **Guideline 5 / locked functionality:** the free package no longer bundles premium activation, license checks, or separate PRO modules. Optional functionality is only available when a **separate add-on plugin** is installed.
> - **Readme disclosure / external services:** the readme now clearly states that the plugin **does not use third-party external services**. It also explains the short, non-blocking **loopback request to this site's own `wp-cron.php`** used to help scheduled tasks run.
> - **Readme structure:** the readme now has a **single Changelog** section.
> - **REST API permissions:** AI routes now use the plugin-prefixed namespace **`museder-restoreone/v1`**, and the REST endpoints use proper **`permission_callback`** checks rather than being publicly writable.
> - **Security / request handling:** the plugin keeps nonce verification, capability checks, sanitization/validation, direct file access guards, and avoids unsafe direct WordPress bootstrap patterns.
>
> Before upload, I checked the code with **Plugin Check 1.7.0** bundled PHPCS ruleset (`plugin-review.xml`), which reported **0 errors and 0 warnings**.
>
> If anything still needs adjustment, I am happy to follow up.
>
> Best regards,
> Jerry Lin
> Author of Museder RestoreOne

https://tw.wordpress.org/plugins/files/2025/11/24_15-13-03_museder-restoreone.zip

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
On Tue, Mar 17, 2026 at 4:58 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

> # 🙌 Thank you for the changes "artherslin"!
>
> **Your plugin is not yet ready to be approved**, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.
>
> Please **check this email thoroughly**, address any issues listed, test your changes, and upload a corrected version of your code if all is well.
>
> ## List of issues found
>
> [隱藏引用文字]
> [隱藏引用文字]
> Example(s) from your plugin:
>
> includes/class-restore-handler.php:299 set_time_limit(600);
> includes/class-backup.php:4037 set_time_limit(0);
>
> ### ## Review: Missing permission_callback in REST API Route
>
> When using register_rest_route() or wp_register_ability() to define custom REST API endpoints, it is crucial to include a proper permission_callback .
>
> 🔒 This callback function ensures that only authorized users can access or modify data through your endpoint.
>
> Code example, checking that the user can change options:
>
> ```
> register_rest_route( 'museder-restoreone/v1', '/my-endpoint', array(
>     'methods' => 'GET',
>     'callback' => 'museder-restoreone_callback_function',
>     'permission_callback' => function() {
>         return current_user_can( 'manage_options' );
>     }
> ) );
> ```
>
> Please check the register_rest_route() documentation and the current_user_can() documentation.
>
> ✅ **When a permission_callback is NOT Required:**

There are valid use cases for public endpoints, such as publicly available data (e.g., posts, public metadata) or endpoints designed for unauthenticated access (e.g., fetching public stats or information).

In these cases, you should use __return_true as the permission_callback to indicate that the endpoint is intentionally public.

🔒 **When a permission_callback IS Required**:

For endpoints that involve sensitive data or actions (e.g., getting not public data, creating, updating, or deleting content).

In these cases, you should always implement proper permission checks.

Possible cases found on this plugin's code:

includes/pro/ai-controller.php:86 register_rest_route(self::NAMESPACE, '/' . self::BASE . '/nl-command', ['methods' => 'POST', 'callback' => [__CLASS__, 'handle_nl_command'], 'permission_callback' => [__CLASS__, 'check_permission']]);
# ↳ Detected: check_permission
# ✨ check_permission checks manage_options but only requires a non-empty nonce value and does not actually verify it (wp_verify_nonce missing).
includes/pro/ai-controller.php:76 register_rest_route(self::NAMESPACE, '/' . self::BASE . '/smart-schedule', ['methods' => 'GET', 'callback' => [__CLASS__, 'handle_smart_schedule'], 'permission_callback' => [__CLASS__, 'check_permission']]);
# ↳ Detected: check_permission
# ✨ check_permission checks manage_options but only requires a non-empty nonce value and does not actually verify it (wp_verify_nonce missing).

[隱藏引用文字]
A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- function musere_save_post(){ ... }
- class MUSERE_Admin { ... }
- update_option('musere_options', $options);
- add_shortcode('musere_shortcode', $callback);
- register_setting('musere_settings', 'musere_user_id', ...);
- define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
- global $musere_options;
- add_action('wp_ajax_**musere**_save_data', ...);
- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of __ (double underscores), wp_ , or _ (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using _n() or __() for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to

break WordPress for your users.

Related to this, using if (!function_exists('NAME')) { around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using if-exists should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2", "musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore", "musederrestoreonereports", "musederrestoreoneadminui" for 167 element(s).

# Looks like there are elements not using common prefixes.
includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

includes/class-backup-jobs.php:963 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:980 add_option($key, $value, '', 'no');

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.
Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✔ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
✔ **I tested my updated plugin on a clean WordPress installation** with WP_DEBUG set to true.

> ⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✔ **I acknowledge that this review will be rejected** if I overlook the issues or fail to test my code.
✔ **I went to "Add your plugin" and uploaded the updated version**. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
✔ I replied to **this** email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

ℹ️ To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us**. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T9 17Mar26/3.8

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
On Tue, Mar 10, 2026 at 6:18 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

> This is an automated message to confirm that we have received your updated plugin file.
>
> File updated by artherslin, version 2.7.243.
> Comment: **Subject:** Museder RestoreOne – Updated version 2.7.243 submitted for review
>
> Dear Plugin Review Team,
>
> Thank you for your previous feedback on Museder RestoreOne. I have addressed the issues raised and am submitting **version 2.7.243** for review.
>
> **Summary of compliance in this version:**
>
> - **Plugin URI** is reachable: https://musederlabs.com/restoreone-plugin/
> - **External services** are documented in the readme (== External services ==): Amazon S3 and OpenAI, with purpose, data sent, when, and links to Terms of Service and Privacy Policy. The plugin does not use external services by default.
> - **Security:** All relevant handlers use nonce verification and capability checks; input is sanitized/validated and output is escaped. Direct file access is guarded with `if ( ! defined( 'ABSPATH' ) ) exit;` where appropriate.
> - **Core loading:** No direct inclusion of wp-load.php, wp-config.php, or wp-blog-header.php. Required core includes (e.g. wp-admin/includes/file.php) are loaded only inside functions when needed.
> - **Paths:** Plugin uses `plugin_dir_path()`, `plugin_dir_url()`, and `wp_upload_dir()` for paths and storage; user-generated files are stored under the uploads directory.

- **PHP limits:** `set_time_limit()` is used only inside long-running functions (backup/restore), not in init or global scope.
- **Input handling:** The plugin no longer processes the whole request body via `file_get_contents('php://input')`; only the necessary fields are read and validated.
- **Options/transients:** Option names use a clear, statically visible prefix (e.g. `museder_restoreone_job_lock_*` via a constant).
- **Plugin Check:** The code has been checked with the Plugin Check plugin's PHPCS ruleset (plugin-review.xml), resulting in **0 errors and 0 warnings**.

I have tested the updated plugin on a clean WordPress installation with WP_DEBUG enabled. The latest code has been uploaded to the plugin directory for your review.

Thank you for your time. I am happy to provide any further information or clarification you need.

Best regards,
Jerry Lin
Author of Museder RestoreOne

https://wordpress.org/plugins/files/2025/11/10_18-18-53_museder-restoreone.zip

--
WordPress Plugins Team | plugins@wordpress.org
https://make.wordpress.org/plugins/
https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/
https://wordpress.org/plugins/plugin-check/
[隱藏引用文字]

> [隱藏引用文字]
> Example(s) from your plugin:
>
> includes/class-restore-handler.php:509 require_once ABSPATH . 'wp-admin/includes/media.php';
>
> [隱藏引用文字]
> A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:
>
> - function musere_save_post(){ ... }
> - class MUSERE_Admin { ... }
> - update_option('musere_options', $options);
> - add_shortcode('musere_shortcode', $callback);
> - register_setting('musere_settings', 'musere_user_id', ...);
> - define('MUSERE_PLUGIN_DIR', plugin_dir_path(__FILE__));
> - global $musere_options;
> - add_action('wp_ajax_**musere**_save_data', ...);
> - namespace artherslin\musederrestoreone;
>
> *Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.*
>
> The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.
>
> You also need to avoid the use of __ (double underscores), wp_ , or _ (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.
>
> Please remember, if you're using _n() or __() for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin!

You don't want to break WordPress for your users.

Related to this, using if (!function_exists('NAME')) { around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using if-exists should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2", "musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore", "musederrestoreonereports", "musederrestoreoneadminui" for 169 element(s).

# Looks like there are elements not using common prefixes.
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');

👉 Continue with the review process.

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.
Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

📋 Complete your checklist.

✓ **I fixed all the issues** in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. **I am familiar with tools** such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
✓ **I tested my updated plugin on a clean WordPress installation** with WP_DEBUG set to true.

> ⚠️ **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

✓ **I acknowledge that this review will be rejected** if I overlook the issues or fail to test my code.
✓ **I went to "Add your plugin" and uploaded the updated version.** *I can continue updating the code there throughout the review process — the team will always check the latest version.*
✓ I replied to **this** email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

ℹ️ To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, **we ask you to thoroughly check all shared issues and fix them before sending the code back to us**. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T8 7Feb26/3.8RC3

[隱藏引用文字]